

Emre Kalem Robot Arm — Troubleshooting Guide

When it will not connect, will not move, or refuses a command

Audience: the person at the bench — knows what a wire is, does not need to know any programming | **Controller:** genuine Arduino Uno R3, VID:PID 2341:0043, currently COM5
Servos: 4× MG996R (base, both shoulders, elbow) + 3× MG90S (wrist pitch, wrist roll, gripper) | **Firmware on the board:** factorylm_arm_controller v1.0.0, build 20260801
Project: C:\RobotArm | **Toolchain:** Arduino IDE 2.3.10 / arduino-cli 1.5.1 / AVR core 1.8.8 / Servo 1.3.0
Primary references: Documentation/SERIAL-PROTOCOL.md and Documentation/V2-SERVO-POWER-AND-WIRING.md
Written: 2026-08-02, the day before the regulated 5 V / 5 A supply arrives

① One-page cheat sheet (the values that absolutely must be right)

What	Value	Source
Servo supply — the new one	5.0 V regulated, 3–5 A	[V2-SERVO-POWER-AND-WIRING §7]
MG996R safe voltage range base D3, shoulders D4+D5, elbow D6	4.8 – 7.2 V	[V2 §7]
MG90S safe voltage range wrist pitch D9, wrist roll D10, gripper D11	4.8 – 6.0 V	[V2 §7]
The old adapter	6.62 V / 700 mA — one MG996R, unloaded, only. Out of spec for every MG90S.	[V2 §7; measured, calibration-log 2026-08-01]
Capacitor across the servo supply	470 – 1000 µF electrolytic, close to the servo, stripe = negative	[V2 §4a]
Inline fuse	1 A, in the supply positive leg	[V2 §3]
Common ground — mandatory	one wire: any Uno GND pin → supply negative rail	[V2, RESOLVED banner 2026-08-01]
What must NEVER be connected	supply + to Uno 5V, VIN, or barrel jack	[V2 §3 rule 3]
Serial settings	115200 baud, 8N1, lines end CR or LF	[SERIAL-PROTOCOL §0]
Proof the right firmware is loaded	OK VER NAME=FACTORYLM-ARM PROTO=1.0 FW=1.0.0	[verified on bench 2026-08-01]
Every joint's limits today	70–110°, CAL=0 — placeholders, not this arm's travel	[joint-limits.csv header]
Watchdog legal values	0 (off, the boot default) or 200 – 10000 ms	[factorylm_arm_controller.ino line 176]
Meter on the signal pin while pulsing	~0.37 V DC (a 1472 µs pulse every 20 ms = 7.4% of 5 V)	[V2 §6; arithmetic shown there]

What	Value	Source
Meter across the servo plug, red ↔ brown	~5 V DC	[V2 §6]
The real emergency stop	the KCD1 rocker switch and the inline fuse	[SERIAL-PROTOCOL §0]

The on-screen E-STOP is not a safety device

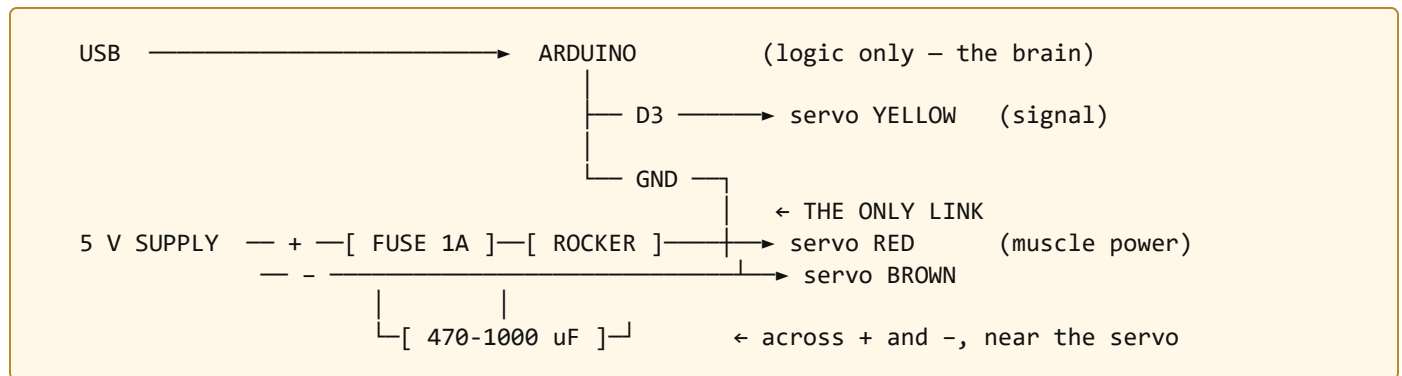
Neither is the Arduino, nor the firmware, nor the browser. A USB cable, a browser tab and a Python program can all die in the middle of a move. The only stop you can rely on is removing servo power at the rocker switch. This is a supervised hobby bench procedure.

⚠ 1 known defect in the shipped console — it will lie to you in exactly one place

1. **A bad watchdog value is reported as a bad joint angle.** If a watchdog timeout outside the legal range is sent, the firmware answers with a bare `ERR E5` carrying no detail [factorylm_arm_controller.ino line 790, `errPlain(F("E5"))`]. The console's `E5` handler is written for a rejected *joint angle*, so it prints *"that starting angle is outside its limits"* [arm-console.html, `ERR_TEXT.E5`]. That sentence is wrong for this case. **Correct meaning:** the watchdog value must be `0`, or between `200` and `10000` ms [ino line 176]. If you were not touching a joint angle when you saw that message, this is what happened.

1 The 60-second mental model

There are **two separate electrical circuits** and **one wire that ties them together**. Nearly every fault on this bench is one of those three things.



The corrected power architecture. [V2 §3]

Circuit one: the USB cable powers the Arduino. That is the thinking part. It is tiny — the Uno's own 5 V pin can supply roughly 500 mA total [V2 §3].

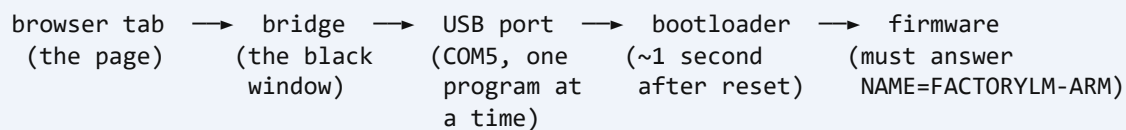
Circuit two: the wall supply powers the servos. That is the muscle. One MG996R can pull up to 2500 mA when stalled [V2 §2] — five times what the whole Arduino can give. This is why the two circuits are kept apart.

The link: one ground wire. Arduino GND to the supply's negative rail, and nothing else crosses between them.

Why this matters: A servo signal is not a thing on its own — it is a voltage *measured against ground*. If the Arduino and the supply do not share a ground, the servo has no reference to measure against, so it cannot read the signal at all. It sits perfectly still and draws nothing. That looks identical to a dead servo, a wrong pin, or bad code — and it cost this project a full session before it was found [V2, RESOLVED banner 2026-08-01].

And on top of that, a chain of five software links

When the arm console says it cannot connect, the failure is in one of these five, and they fail in this order:



The console deliberately refuses to talk to a board that does not identify itself correctly. It opens the port, waits two seconds for the bootloader to finish, then asks `VER` up to three times. Unless the answer contains exactly `NAME=FACTORYLM-ARM`, it stops and tells you what it actually found [SERIAL-PROTOCOL §6].

Why this matters: That refusal is a feature. Silently talking to the wrong program on the board is how you end up sending a joint command to a sketch that interprets it as something else entirely.

2 First power-up with the new 5 V / 5 A supply

Do this once, in this order, the first time the new supply is used. It takes about fifteen minutes and it is the difference between finding a wiring error with a meter and finding it with smoke.

Before anything is switched on

Set the supply to **5.0 V** if it is adjustable, and confirm it with a meter *before* a servo is connected to it. An adjustable supply left at 12 V destroys every servo on this arm in about a second.

Step by step

1. ☐ **Rocker OFF. Supply unplugged from the wall.** Nothing energised.
2. ☐ **Wire the supply into the barrel jack adapter** — the 5.5 × 2.1 mm screw-terminal type. Centre pin is + on virtually every one of these; confirm with a meter rather than assuming.
3. ☐ **Fit the 1 A fuse holder in the positive leg**, between the supply and the rocker [V2 §3].
4. ☐ **Fit the 470–1000 µF capacitor across + and –**, physically close to the servo. The stripe on the capacitor body is the negative side. Backwards is a bang [V2 §4a].
5. ☐ **Run servo power off the breadboard.** Servo + and – go from the screw terminals straight to the servo. Breadboard spring contacts are rated about 1 A at best and degrade with use; a poor contact there produces intermittent faults that look exactly like a failing servo [V2 §4b]. Signal wires through the breadboard are fine.
6. ☐ **Fit the common ground** — any Uno GND pin to the supply negative rail.
7. ☐ **Plug in the supply, rocker still OFF. Meter across the supply output.**

✓**You should see:** between 4.9 and 5.1 V DC. If it reads 6.6 V you are still on the old adapter. If it reads 12 V, stop and adjust before going further.

8. ☐ **Rocker ON, no servo connected yet.** Meter at the far end of the servo power leads.

✓**You should see:** the same ~5 V at the servo end. A big drop here means a bad crimp or a breadboard contact still in the path.

9. ☐ **Rocker OFF. Connect exactly one servo** — the wrist roll, D10, is the right first choice: it is an MG90S with the least mechanical load of all seven [wiring-map.csv, servo type DOC-CONFIRMED], and 5 V is comfortably inside its 4.8–6.0 V range.
10. ☐ **Horn off.** Tape a toothpick or a paper flag to the spline so you can see small movements [V2 §5].
11. ☐ **Rocker ON, then enable that one joint in the console** and nudge it.

✓**You should see:** the flag moves, the board does not reset, and the console stays connected. A reset here means a brownout — check the capacitor and the ground.

One joint at a time, even on 5 A

5 A is enough for several servos in normal motion, but the arm is being reassembled with joints at unknown angles and no measured limits. The reason to go one at a time now is mechanical, not electrical: a joint driven into a hard stop stalls, and a stalled MG996R pulls about 2.5 A on its own [V2 §2].

3 “It will not connect” — work down this list

What you see	What it means	Fix
The dropdown has no COM port in it	Windows cannot see the board at all	Check the USB cable is a data cable, not charge-only. Try a different socket. The genuine Uno shows as <code>USB Serial Device</code> with hardware id <code>2341:0043</code> .
“Port is blocked” / access denied	Another program already holds the port. Only one can	Close the Arduino IDE Serial Monitor, <code>SERVO_CONSOLE.bat</code> , and any browser tab that has ever connected. A tab using Web Serial keeps the port until the tab is closed — minimising it is not enough <i>[verified on bench 2026-08-01]</i> .
It connects, then reports the board is not the arm controller, quoting something odd	The wrong program is on the board	See §4 below. This is what happened on 2026-08-01 and it is the single most likely cause.
The black console window flashes and vanishes	The bridge could not start	Read what it printed. Usually it is Python: only Python 3.11 on this laptop has <code>pyserial</code> installed. <code>py -3</code> points at 3.14, which does not <i>[START ARM GUI.bat, header comment]</i> .
The page loads but every button does nothing	The bridge behind it has stopped	The black window must stay open the whole time. Closing it disconnects the board.
No Python at all	—	You do not need it. Open <code>Software\arm-console\arm-console.html</code> directly in Chrome or Edge and use CHOOSE PORT. Firefox will not work — it has no Web Serial.

Prove where the block is, in one command

Double-clicking `START ARM GUI.bat` runs a preflight first. It clears any leftover bridge and then *names* whatever else is holding the port, instead of leaving you to guess.

✓**You should see:** `COM5 is free.` — anything else, it will tell you which program to close.

A trap for anyone scripting this later

The file is called `START ARM GUI.bat`, and Windows' command processor reads a leading `START` as its own built-in command. Launching it from a script therefore does nothing at all, silently. The desktop icon is unaffected. Any launcher must use PowerShell's `Start-Process` *[verified on bench 2026-08-02]*.

4 The wrong program on the board — how to tell, how to fix

Two different programs get loaded onto this Uno, and only one of them can talk to the arm console.

Program	What it is for	How it announces itself
<code>factorylm_arm_controller</code>	The real one. Six joints, the arm console, the whole protocol	RDY NAME=FACTORYLM-ARM PROTO=1.0 FW=1.0.0
<code>emre_kalem_single_servo_bench_test</code>	The one-servo keyboard tester, used for calibrating a joint on its own	EMRE KALEM ARM - SINGLE SERVO BENCH TEST

If the bench-test program is loaded and the console asks it `VER`, it answers one character at a time, because it is a keyboard console and reads every letter as a separate keystroke:

```
[stop] unknown command 'V'. Type 'h' for help.  
[stop] unknown command 'E'. Type 'h' for help.  
[stop] unknown command 'R'. Type 'h' for help.
```

The exact symptom recorded on 2026-08-01. [verified on bench 2026-08-01]

Loading the arm controller

Rocker OFF and the supply unplugged first — loading a program restarts the board, and a powered servo can twitch when that happens. Then, in a terminal:

```
"C:\Program Files\Arduino CLI\arduino-cli.exe" compile --fqbn arduino:avr:uno "C:\RobotArm\Software\fact  
"C:\Program Files\Arduino CLI\arduino-cli.exe" upload -p COM5 --fqbn arduino:avr:uno --verify "C:\RobotA
```

✓**You should see:** compile reports Sketch uses 11650 bytes (36%) and Global variables use 405 bytes (19%) , and upload reports New upload port: COM5 (serial) . *[verified on bench 2026-08-02]*

Going back to the bench-test program

Same command, different folder. You will want this every time you calibrate a joint on its own:

```
"C:\Program Files\Arduino CLI\arduino-cli.exe" upload -p COM5 --fqbn arduino:avr:uno --verify "C:\RobotA
```

“No error” is not proof it worked

An upload that prints nothing alarming can still have gone nowhere. The only proof is that the board *identifies itself* afterwards. Open the arm console and connect: if the header goes from DISCONNECTED to connected and six joint cards appear, the right program is running. That is a stronger check than any readback.

5 What the refusal messages mean

The controller never fails silently. Every rejected command comes back with a code, and the console turns it into a sentence. Here is what is behind each one *[SERIAL-PROTOCOL §7 error table]*.

Code	What it really means	What to do
E4 with RESERVED=shoulder_pair	You addressed joint 2. It does not exist as a control — it is the shoulder's <i>second</i> servo, and it is only ever driven together with the first, as J1	Use J1. Nothing to fix.
E5 with REQ=... MIN=... MAX=...	The angle you said the joint is sitting at is outside that joint's limits. Every joint is still stuck at the 70–110 placeholder	If the joint really is out there, measure it and widen its limits in <code>joint-limits.csv</code> , then press LOAD LIMITS FILE. Do not guess the numbers.
E5 with nothing after it	Not an angle problem. This is the watchdog defect from page 1	The watchdog value must be 0, or 200–10000 ms.
E6	That joint is already enabled, or is not enabled and you tried to move it	Look at the card — it says DISABLED or not.
E7	The E-STOP is latched. Every enable and move will be refused until it is cleared	Press CLEAR E-STOP, then enable each joint again with a fresh angle. This is a common cause of “nothing works any more”.
E9	You tried to change limits, speed or the mirror on a joint that is currently enabled	Disable it first. You cannot move the goalposts under a driven joint.
E10	A row in <code>joint-limits.csv</code> is illegal — limits must sit inside 0–180° and min must be below max	Fix the row, reload the file.
E11	The shoulder mirror setting is illegal, or the offset would swing the second servo past its travel	Reduce <code>mirror_offset_deg</code> or narrow the shoulder's limits.
E12	Speed outside 1–90 °/s	Pick a number in range.
E13	The shoulder is locked on purpose. It will not drive D4 and D5 until it knows whether the second servo turns the same way as the first or the opposite way	Measure the mirror — see §7. Never guess it.

Why this matters: Codes E10, E11 and E12 exist because they were once all E5, and the console was telling the operator “that starting angle is outside this joint's limits” for three completely different problems with three different remedies *[arm-console.html, comment above ERR_TEXT]*. A message that names the wrong cause is worse than no message.

6 It connects, but a servo will not move

Work this table top to bottom. The order is deliberate — the top entries are cheap to check and have actually happened on this bench.

Symptom	Likely cause	Fix / how to confirm
Nothing moves. No noise, no heat, no board reset, no matter which pin	Missing common ground. The number one cause on this project	One wire: Uno <code>GND</code> → supply negative. Silence with power present <i>is</i> the symptom — a servo with no ground reference draws nothing [V2, <i>RESOLVED banner</i>].
Nothing moves on one pin, but another pin works	That signal wire, that breadboard row, or that servo	Move the suspect servo's signal wire onto a pin you know works and drive that joint instead. Moves there = the servo is fine and the fault is the wire or the pin. Still dead = the servo.
The board resets or the console disconnects the moment a servo starts	Brownout — the supply cannot deliver the startup surge	Fit the capacitor if it is not fitted [V2 §4a]. Get servo current off the breadboard [V2 §4b]. Check you are on the 5 A supply and not the 700 mA one.
It moves, but only a little, and stops well short	Nothing is wrong. Every joint is clamped to the <code>70–110°</code> placeholder — 40° of travel	Measure that joint's real travel and widen its row in <code>joint-limits.csv</code> . See §7.
The servo turns, but the arm does not	The horn is off, or the joint is not bolted to it	Expected while the arm is apart. Not a fault.
Something moves, but not the joint named on the card	The pin-to-motor map is wrong for that channel	See §7 — this is a known open question on D3.
Buzzing, getting hot, not moving	Stalled against a mechanical limit, or two servos fighting	Rocker OFF immediately. A stalled MG996R pulls about 2.5 A and will cook [V2 §2]. Find the obstruction before re-enabling.
Erratic, intermittent, works then does not	A contact — usually a breadboard spring carrying servo current	Run servo power on screw terminals straight to the servo [V2 §4b]. Also check the mini-breadboard's <code>+</code> / <code>-</code> rails are not split in the middle; on many they are.
Everything is refused with <code>E7</code>	The E-STOP latch is still set	Press CLEAR E-STOP, then re-enable each joint with a fresh angle.
Everything went limp on its own and the console said the watchdog tripped	The host stopped talking — tab closed, bridge stopped, cable knocked	Reconnect, clear, re-enable. The watchdog is off by default at boot, so this only happens if it was switched on.

The meter test that ends the guessing

With one joint enabled and holding, probe two places [V2 §6]:

Probe	Expect	What a wrong reading means
Signal pin ↔ <code>GND</code>	<code>~0.37 V DC</code>	<code>0 V</code> = no pulses on that pin at all. <code>~5 V</code> = stuck high.
Servo plug RED ↔ BROWN	<code>~5 V DC</code>	<code>0 V</code> = that servo has no power. Prime suspect.

Why this matters: 0.37 V looks like a broken reading and is not. The signal is a 1472 μs pulse repeated every 20 ms — a 7.4% duty cycle — and 7.4% of 5 V is 0.37 V. A cheap meter averaging 50 Hz will wander. What matters is the difference between *about a third of a volt*, *zero*, and *about five*.

7 Three things nobody has measured yet

These are not faults. They are blanks, and the software is deliberately refusing to fill them in for you. Filling a blank with a guess is how a joint gets driven into a hard stop.

a. Every joint's travel limits

All six joints read `70–110°` with `calibrated=no` *[joint-limits.csv]*. That is the same conservative default the bench-test sketch uses. It is **not** the travel of any joint on this arm.

To fill one in: load the bench-test program, test that joint alone with the horn off, find where it genuinely stops each way, back off a few degrees from each end, and write those two numbers into its row. Then set `calibrated=yes`, log it in `Calibration_Notes/calibration-log.csv`, and press LOAD LIMITS FILE in the console.

Only ever set calibrated=yes for a joint you personally measured

A row still sitting at 70–110 with `calibrated=yes` is a lie the console will believe and repeat, and the amber UNCALIBRATED badge is the only thing that would have warned you *[joint-limits.csv header]*.

The elbow (D6) is the lightest of the four MG996R joints and a sensible one to calibrate first *[joint-limits.csv, joint_id=3 notes]*.

b. Which motor is physically on D3

Two project files disagree, and both are in the repository:

File	Says D3 is
<code>Software/wiring-map.csv</code> , index 0	Base — an MG995/MG996R, marked <code>INFERRED</code>
<code>Calibration_Notes/calibration-log.csv</code> , 2026-08-01	"the servo physically wired to D3 was the GRIPPER/CLAW"

The console shows this contradiction on the J0 card rather than picking a side. Settle it **by size, with no power connected at all**: follow the signal wire out of D3 and look at the servo it lands on.

If the servo is	Then	Correct
Large, heavy, metal gear horn (MG996R, ~55 g)	D3 really is a base-class servo — the wiring map is right	the calibration-log note
Small, light, plastic (MG90S, ~14 g)	D3 is the gripper — the bench log is right	<code>wiring-map.csv</code> and <code>joint-limits.csv</code> , together

Change both files or neither

`wiring-map.csv` and `joint-limits.csv` use the same joint numbering on purpose, so that this project has exactly one numbering system *[SERIAL-PROTOCOL §1]*. Correcting one and not the other re-creates the confusion that produced this contradiction in the first place.

c. The shoulder mirror

J1 drives two MG996Rs (D4 and D5) through one link. The firmware refuses to enable it at all until it is told whether the second servo must be given the *same* angle or the *opposite* one — that is the `E13` refusal.

This is the one measurement never to guess

Get it wrong and two MG996Rs push against each other through a single link. They will hold, get hot, and strip a gear. It is the highest-consequence mechanical failure available on this arm, which is why the software removes the ability to express it rather than warning about it [*SERIAL-PROTOCOL §1, "Why ID 2 is reserved"*].

To measure it: with the shoulder linkage **unbolted** and both horns off, use the bench-test sketch on D4 and D5 **separately, never paired**. Note whether the two output shafts point the same physical direction when given the same angle (that is `same`) or when given mirrored angles (that is `inverted`). Write it into the `mirror_mode` column of the shoulder row.

If it turns out to be `inverted`, there is a second number to measure — the offset. Command each servo until the two shafts point the same physical way; call those angles *a* and *b*; then `mirror_offset_deg = (a + b) / 2 - 90` [*joint-limits.csv header*].

Why this matters: Servo horns fit on a splined shaft in whole-tooth steps — about 18° apart on an MG996R — so two horns fitted perfectly symmetrically is the exception, not the rule. If the true mirror axis is 96° and the offset is left at 0, the two servos fight each other by 12° for the entire time joint 1 is enabled.

8 Two faults already paid for — do not re-diagnose these

Both of these cost a working session. If a symptom below matches, go straight to the fix.

1. The servo that had everything and still would not move

Power present. Valid $\sim 1472\ \mu\text{s}$ pulses on D3. Twenty-one attach events across seven pins. No current draw, no brownout, no movement. **There was no wire between Arduino GND and the supply negative.** The complete absence of any electrical event was the diagnostic clue — a servo that is trying and failing makes noise and draws current; one with no ground reference does neither *[V2, RESOLVED banner 2026-08-01]*.

2. “The port is blocked”

Only one program at a time can hold a serial port, and four things here want it: the bridge, the Arduino IDE Serial Monitor, a browser tab using Web Serial, and `SERVO_CONSOLE.bat`. **A browser tab keeps the port until the tab itself is closed** — not minimised, not navigated away from. Closed. The preflight now names whatever is holding it *[verified on bench 2026-08-01]*.

And two tool traps worth knowing

- Windows reports **phantom COM ports** from stale registry entries. The port list you can trust comes from the device enumeration, not the registry — which is why the preflight and the bridge both use it *[verified on bench 2026-08-01]*.
- A program that prints raw text coming back from the board can crash on an odd byte, because the Windows console is not set to UTF-8 by default. The shipped bridge is safe from this — it converts the text properly before it ever reaches a console *[arm-bridge.py lines 380 and 493, verified on bench 2026-08-02]*. Anything hand-written to talk to the board needs the same care.

Rules that must survive everything

- Nothing is attached at boot, and no joint is ever centred automatically. Every enable requires you to say where the joint is sitting *first*, so the very first pulse a servo receives equals its current position and nothing jumps.
- Limits are **measured data**. Never invented, never inferred, never copied from another arm.
- D4 and D5 are one joint. Always.
- **HOLD is not E-STOP**. HOLD freezes motion with the joints still powered and holding position. E-STOP cuts the signal — the joints go limp, and a loaded arm sags under its own weight.
- MIRA-side rule for the whole project: read-only intelligence. This firmware answers; it does not act on its own.

The last word on stopping

The rocker switch and the inline fuse are the emergency stop. The button on the screen, the Arduino, the firmware and the browser are conveniences. If something is moving that should not be, reach for the switch — not the mouse.